Bifrost — Live Data Bridge

Mimir Labs Technical Publication

Document:	ML-WP-010	**Version:**	1.1
Author:	Christopher Gaither	**Date:**	April 2026
Classification:	Public		

[# Bifrost — Live Data Bridge White Paper

Mimir Labs Technical Publication Document Version: 0.1 (Evolving — Pre-Development) **Date:** March 2026 **Classification:** Public **Status:** Planned (YGGDATA-312, 13 stories, all To Do)

0.1 Executive Summary

Bifrost is a standalone, persistent background service that maintains live data synchronization between multiple enterprise systems. It supports three sync modes — bidirectional, mirror (digital twin), and unidirectional (pilot) — enabling a progressive customer lifecycle from proof-of-concept to full production.

Bifrost uses the Mimisbrunnr semantic model as its Rosetta Stone — a universal reference schema of 300+ tables across 17 business domains that provides the shared vocabulary for routing and translating data between systems. While Yggdrasil ERP is the default hub, Bifrost is system-agnostic: it can synchronize data between any combination of connected systems (Salesforce, Business Central, Smartsheet, PostgreSQL, etc.) using Mimisbrunnr as the semantic mediation layer.

Bifrost is the third and final tool in the Mimir Labs data platform trilogy. Where Ratatosk discovers and Ragnarok migrates, Bifrost keeps systems in continuous harmony.

This is an evolving document. Bifrost is not yet implemented. Content reflects the planned architecture as defined in the YGGDATA-312 epic and its 13 child stories.

0.2 1. Platform Position

Tool	Role	Lifecycle	License
Ratatosk	Migration Scouter	One-time	Migration-Key
Ragnarok	Migration Executor	One-time	Migration-Key
Bifrost	Live Data Bridge	Ongoing	Subscription

```
Ratatosk (.ratatosk.json) ──→ Ragnarok (one-time bulk migration)
                          └──→ Bifrost  (persistent live sync)
```

Bifrost packages the **mapping intelligence** from Ratatosk manifests and the **execution patterns** from Ragnarok into a persistent, event-driven synchronization engine. Where Ragnarok performs a one-time migration, Bifrost keeps systems in continuous harmony.

0.3 2. Planned Technology Stack

Component	Specification
Language	C++17
Framework	Qt 6.2+ (Core, Network, Sql)
Database	SQLite (local sync state), libpq (PostgreSQL listener)
Encryption	OpenSSL (AES-256 credential vault)
Build System	CMake 3.16+
Deployment	Single executable + config file, no container orchestration

0.3.1 2.1 Design Constraints

- **Standalone deployment** — Single executable, no external orchestration required
- **No AI/LLM** — 100% deterministic logic for field ownership rules and conflict resolution
- **Local-first security** — Encrypted vault (AES-256) for all API keys and OAuth tokens
- **No cloud dependency** — Runs on customer premises or Mimir Labs VPS

0.4 3. Sync Modes

Bifrost operates in one of three sync modes, configurable globally or per-route. Each mode maps to a specific phase of the customer lifecycle:

0.4.1 3.1 Mirror (One-Way, Read-Only Target) — Pilot

"Show me what my data looks like in the target system."

- Source → Target only. No write-back to source.
- Target receives updates but never pushes changes back.
- Conflict detection is **disabled** — source is always authoritative.
- Local target modifications are **overwritten** on next sync.
- **Use case:** Proving Bifrost works. Zero risk to incumbent systems. If something goes wrong, turn Bifrost off — the incumbent is untouched.

0.4.2 3.2 Unidirectional (One-Way, Target Preserved) — Target Pilot

"Let me start working in the target system while my incumbents keep feeding data in."

- Source changes propagate to target, but target changes are **never synced back**.
- Target-local changes are **preserved** — Bifrost will not overwrite data entered directly into the target.
- **Use case:** Customer is trialing a new system (e.g., Yggdrasil ERP) as their working platform. Incumbent data keeps flowing in, but the customer also creates new records and runs workflows directly in the target.

0.4.3 3.3 Bidirectional (Default) — Production

"Keep everything in sync across all connected systems."

- Changes in any system propagate to all other linked systems.
- Conflict detection and resolution active (golden record logic).
- Full fan-out: N-1 outbound writes per inbound event.
- **Use case:** Full production. All connected systems are live and authoritative for their designated fields.

0.4.4 3.4 Lifecycle Progression

```
Mirror (prove it works)  →  Unidirectional (start working)  →  Bidirectional (go live)
    Bifrost pilot              Target system pilot                  Production
```

Mode transitions happen without data loss or service restart.

0.5 4. Planned Architecture

0.5.1 4.1 Manifest Consumption (YGGDATA-315)

Bifrost consumes `.ratatosk.json` manifests as its foundational routing table:

- Each manifest entry (table + column + business label + taxonomy group) maps to a sync route
- Configuration layer on top of manifest entries: sync mode, TTL, priority source, conflict policy
- Manifest versioning — detects when a new manifest supersedes an existing one

0.5.2 4.2 Event Listeners

Four event listeners detect changes in connected systems:

Listener	System	Protocol	Story
`PostgreSQLListener`	PostgreSQL (including Yggdrasil)	LISTEN/NOTIFY triggers	YGGDATA-317
`SalesforceListener`	Salesforce	Outbound Messages, Platform Events, CDC	YGGDATA-318
`BCListener`	Business Central	OData v4 change tracking, polling	YGGDATA-319
`SmartsheetListener`	Smartsheet	Events API, webhook callbacks	YGGDATA-320

Each listener: - Detects field-level changes (not just record-level) - Enqueues change events for the sync engine - Handles rate limiting and authentication for its target system - Supports Tier A (live connection) and Tier B (replay from JSON capture file)

0.5.3 4.3 Sync Engine (YGGDATA-321)

The fan-out sync engine is the central event dispatcher:

```
Inbound Event (any listener)
  → SyncEvent queue
  → RoutingTable lookup (from manifest)
  → Golden Record check (bidirectional only)
  → Fan-out: N-1 outbound writes to all other connected systems
```

Key capabilities: - **Cycle detection** — Tags every outbound write with a `bifrost_correlation_id`. Inbound events carrying a known correlation ID are silently dropped. - **Causal ordering** — Sequence numbers per entity ensure update order is preserved. - **Idempotency** — Deduplication by (entity_id, field, timestamp) tuple. - **Parallel fan-out** — Thread pool for concurrent writes across systems. - **Dry-run mode** — Log planned writes without executing (Tier B).

0.5.4 4.4 Golden Record Logic (YGGDATA-322)

Deterministic source-of-truth rules per field (bidirectional mode only):

- **Field ownership model** — Each field has a designated `owner_system`
- **Owner writes** are always propagated immediately (fast path)
- **Non-owner writes** are checked against watermark for conflicts

Conflict policies:

Policy	Behavior
`LAST_WRITE_WINS`	Compare timestamps, newer value wins
`SOURCE_PRIORITY`	Owner system always wins, discard non-owner change
`PAUSE_AND_ALERT`	Halt sync for this (entity, field) pair, notify human reviewer

0.5.5 4.5 Conflict Resolution UI (YGGDATA-323)

Qt Widgets dialog for human conflict resolution:

- Conflict list view (sortable by entity, field, age, system)
- Side-by-side value comparison with timestamps
- Resolution actions: Accept A, Accept B, Merge (custom value), Skip
- Batch operations: "Accept All Owner" for bulk resolution
- Audit trail: every resolution logged with operator, timestamp, reason
- System tray notification on new conflicts

0.5.6 4.6 Sync State Database (YGGDATA-316)

SQLite-backed local persistence:

Table	Purpose
`sync_watermarks`	Last sync timestamp per (system, entity, field)
`pending_changes`	Queued changes with retry count and next_retry_at
`conflict_log`	Detected conflicts with competing values and resolution status
`resolution_audit`	Human resolution decisions
`listener_health`	Heartbeat, last event, error count per listener
`sync_metrics`	Aggregate throughput, latency, error rates
`dead_letters`	Changes that exceeded max retries

Retry logic: exponential backoff (1s → 2s → 4s → … max 5 minutes), configurable max retries, WAL mode for crash safety.

0.6 5. Planned Credential Security (YGGDATA-314)

Encrypted credential vault:

- **Algorithm** — AES-256-GCM
- **Stored credentials** — OAuth tokens (Salesforce, BC), API keys (Smartsheet), database passwords (PostgreSQL targets)
- **Vault key** — Derived from operator passphrase via PBKDF2
- **No plaintext** — Credentials never written to disk in plaintext
- **Rotation** — Vault supports credential rotation without service restart

0.7 6. Observability (YGGDATA-324)

0.7.1 6.1 Health Monitoring

- **CLI:** `./Bifrost --health` returns status + exit code (0=healthy, 1=degraded, 2=critical)
- **HTTP:** `/health` and `/metrics` endpoints (Prometheus-compatible)
- **System tray:** Green/yellow/red icon based on aggregate health

Health is computed from: listener liveness, pending queue depth, drift entity count, unresolved conflict count, dead letter queue size.

0.7.2 6.2 Structured Logging

- JSON log output with severity, timestamp, component, correlation_id
- Log rotation: 10MB per file, 10 files max
- Correlation IDs enable end-to-end event tracing across systems

0.7.3 6.3 Sync Status

Per-entity sync observability:

Status	Meaning
In Sync	All systems agree, last sync within TTL
Pending	Change detected, propagation queued

Status	Meaning
Drift Detected	Values differ across systems beyond TTL

0.7.4 6.4 Daily Reconciliation

Automated daily job (configurable schedule): - Compares row counts and checksums across all linked systems - Identifies drifted records - Outputs structured report (JSON + human-readable) - Optional email delivery

0.8 7. Dual-Mode Operation

0.8.1 7.1 Tier A — Live Sync (Production)

Active connections to all incumbent systems with real-time event listeners, data propagation, conflict detection, and persistent sync state.

0.8.2 7.2 Tier B — Schema + Configuration Review (Dry-Run)

- Load schemas from DDL files, migration scripts, or manifests
- Validate routing configuration without live connections
- Generate sync route maps
- Simulate sync events and log planned actions
- **Use cases:** Pre-deployment validation, CI/CD integration, customer demos

0.9 8. Performance Targets

Metric	Target
Sync throughput	≥ 100 record syncs/minute sustained
Event detection latency	< 1 second (LISTEN/NOTIFY)
Fan-out latency	Within configured TTL per route
72-hour soak test	Zero unresolved conflicts (except deliberate)

0.10 9. Implementation Roadmap

Story	Title	Status
YGGDATA-313	Project scaffold & CMake build system	To Do
YGGDATA-314	Encrypted credential vault (AES-256)	To Do
YGGDATA-315	Manifest routing table & sync configuration	To Do
YGGDATA-316	Sync state database (SQLite)	To Do
YGGDATA-317	PostgreSQL listener (LISTEN/NOTIFY)	To Do
YGGDATA-318	Salesforce listener (webhook receiver)	To Do
YGGDATA-319	Business Central listener (OData polling)	To Do
YGGDATA-320	Smartsheet listener (Events API)	To Do
YGGDATA-321	Fan-out sync engine & cycle detection	To Do
YGGDATA-322	Golden record logic & conflict detection	To Do
YGGDATA-323	Conflict resolution UI	To Do
YGGDATA-324	Health monitoring & status dashboard	To Do
YGGDATA-325	3-way integration test (SF + BC + Smartsheet)	To Do

0.11 10. Definition of Done

- Successful 3-way bidirectional sync (SF + BC + Smartsheet) for 72 hours
- Mirror mode: source changes propagate, target changes overwritten, no write-back
- Unidirectional mode: source changes propagate, target-local changes preserved
- Mode transition (Mirror → Unidirectional → Bidirectional) with zero data loss
- Conflict UI: detection, pause, human resolution, sync resumption
- `.ratatosk.json` manifest loaded and used as routing table
- All credentials encrypted at rest
- Cycle detection verified
- Per-entity Last Synced At timestamps accurate
- Daily reconciliation report runs on schedule
- Throughput ≥ 100 syncs/minute sustained